

extremely common (Li et al., 2019). As we have seen, locking is not just hard, but also expensive and operating systems have adopted RCU to avoid locking altogether (McKenney et al., 2013).

An active research area deals with recording and replaying a process' execution (Viennot et al., 2013). Replaying helps developers track down hard-to-find bugs and security experts to investigate incidents.

Speaking of security, a major event in 2018 was the disclosure of a series of very serious security vulnerabilities in modern CPUs. They required changes everywhere: hardware, firmware, operating system, and even applications. For this chapter, the scheduling implications are especially relevant. For instance, Windows adopted a scheduling algorithm to prevent code in different security domains from sharing the same processor core (Microsoft, 2018).

Scheduling (both uniprocessor and multiprocessor) in general is still a topic near and dear to the heart of some researchers. Some topics being researched include scheduling in clusters for deep learning (Xiao et al., 2018), scheduling for microservices (Sriraman, 2018), and schedulability (Yang et al., 2018). All in all, processes, threads, and scheduling are not hot topics for research as they once were. The research has moved on to topics like power management, virtualization, clouds, and security.

2.7 SUMMARY

To hide the effects of interrupts, operating systems provide a conceptual model consisting of sequential processes running in parallel. Processes can be created and terminated dynamically. Each process has its own address space.

For some applications, it is useful to have multiple threads of control within a single process. These threads are scheduled independently and each one has its own stack, but all the threads in a process share a common address space. Threads can be implemented in user space or in the kernel.

Alternatively, high-throughput servers may opt for an event-driven model instead. Here, the server operates as a finite state machine that responds to events and interacts with the operating system using non-blocking system calls.

Processes can synchronize with one another using synchronization and interprocess communication primitives, for example, semaphores, monitors, or messages. These primitives are used to ensure that no two processes are ever in their critical regions at the same time, a situation that leads to chaos. A process can be running, runnable, or blocked and can change state when it or another process executes one of the interprocess communication primitives. Interthread communication is similar.

A great many scheduling algorithms have been studied. Some of these are primarily used for batch systems, such as shortest-job-first scheduling. Others are common in both batch systems and interactive systems. These algorithms include

round robin, priority scheduling, multilevel queues, guaranteed scheduling, lottery scheduling, and fair-share scheduling. Some systems make a clean separation between the scheduling mechanism and the scheduling policy, which allows users to have control of the scheduling algorithm.

PROBLEMS

1. In Fig. 2-2, three process states are shown. In theory, with three states, there could be six transitions, two out of each state. However, only four transitions are shown. Are there any circumstances in which either or both of the missing transitions might occur?
2. Suppose that you were to design an advanced computer architecture that did process switching in hardware, instead of having interrupts. What information would the CPU need? Describe how the hardware process switching might work.
3. On all current computers, at least part of the interrupt handlers are written in assembly language. Why?
4. When an interrupt or a system call transfers control to the operating system, a kernel stack area separate from the stack of the interrupted process is generally used. Why?
5. A computer system has enough room to hold four programs in its main memory. These programs are idle waiting for I/O half the time. What fraction of the CPU time is wasted?
6. A computer has 2 GB of RAM of which the operating system occupies 256 MB. The processes are all 128 MB (for simplicity) and have the same characteristics. If the goal is 99% CPU utilization, what is the maximum I/O wait that can be tolerated?
7. Multiple jobs can run in parallel and finish faster than if they had run sequentially. Suppose that two jobs, each needing 10 minutes of CPU time, start simultaneously. How long will the last one take to complete if they run sequentially? How long if they run in parallel? Assume 50% I/O wait.
8. Consider a multiprogrammed system with degree of 5 (i.e., five programs in memory at the same time). Assume that each process spends 40% of its time waiting for I/O. What will be the CPU utilization?
9. Explain how a Web browser can utilize the concept of threads to improve performance.
10. Assume that you are trying to download a large 2-GB file from the Internet. The file is available from a set of mirror servers, each of which can deliver a subset of the file's bytes; assume that a given request specifies the starting and ending bytes of the file. Explain how you might use threads to improve the download time.
11. In the text it was stated that the model of Fig. 2-10(a) was not suited to a file server using a cache in memory. Why not? Could each process have its own cache?

12. In Fig. 2-8, a multithreaded Web server is shown. If the only way to read from a file is the normal blocking read system call, do you think user-level threads or kernel-level threads are being used for the Web server? Why?
13. In the text, we described a multithreaded Web server, showing why it is better than a single-threaded server and a finite-state machine server. Are there any circumstances in which a single-threaded server might be better? Give an example.
14. In Fig. 2-11 the register set is listed as a per-thread rather than a per-process item. Why? After all, the machine has only one set of registers.
15. Why would a thread ever voluntarily give up the CPU by calling *thread_yield*? After all, since there is no periodic clock interrupt, it may never get the CPU back.
16. In this problem, you are to compare reading a file using a single-threaded file server and a multithreaded server. It takes 15 msec to get a request for work, dispatch it, and do the rest of the necessary processing, assuming that the data needed are in the block cache. If a disk operation is needed, as is the case one-third of the time, an additional 75 msec is required, during which time the thread sleeps. How many requests/sec can the server handle if it is single threaded? If it is multithreaded?
17. What is the biggest advantage of implementing threads in user space? What is the biggest disadvantage?
18. In Fig. 2-14 the thread creations and messages printed by the threads are interleaved at random. Is there a way to force the order to be strictly thread 1 created, thread 1 prints message, thread 1 exits, thread 2 created, thread 2 prints message, thread 2 exits, and so on? If so, how? If not, why not?
19. Suppose that a program has two threads, each executing the *get_account* function, shown below. Identify a race condition in this code.

```
int accounts[LIMIT]; int account_count = 0;

void *get_account(void *tid) {
    char *lineptr = NULL;
    size_t len = 0;

    while (account_count < LIMIT)
    {
        // Read user input from terminal and store it in lineptr
        getline(&lineptr, &len, stdin);

        // Convert user input to integer
        // Assume user entered valid integer value
        int entered_account = atoi(lineptr);

        accounts[account_count] = entered_account;
        account_count++;
    }
}
```

```
// Deallocate memory that was allocated by getline call
free(lineptr);
return NULL; }
```

20. In the discussion on global variables in threads, we used a procedure *create_global* to allocate storage for a pointer to the variable, rather than the variable itself. Is this essential, or could the procedures work with the values themselves just as well?
21. Consider a system in which threads are implemented entirely in user space, with the run-time system getting a clock interrupt once a second. Suppose that a clock interrupt occurs exactly while some thread executing in the run-time system is at the point of blocking or unblocking a thread. What problem might occur? Can you solve it?
22. Suppose that an operating system does not have anything like the *select* system call to see in advance if it is safe to read from a file, pipe, or device, but it does allow alarm clocks (timers) to be set that interrupt blocked system calls. Is it possible to implement in user space a threads package that will not block all threads when one thread performs a system call that may block? Explain your answer.
23. Does Peterson's solution to the mutual-exclusion problem shown in Fig. 2-24 work when process scheduling is preemptive? How about when it is nonpreemptive?
24. Can the priority inversion problem discussed in Sec. 2.3.4 happen with user-level threads? Why or why not?
25. In Sec. 2.3.4, a situation with a high-priority process, *H*, and a low-priority process, *L*, was described, which led to *H* looping forever. Does the same problem occur if round-robin scheduling is used instead of priority scheduling? Discuss.
26. In a system with threads, is there one stack per thread or one stack per process when user-level threads are used? What about when kernel-level threads are used? Explain.
27. What is a race condition?
28. When a computer is being developed, it is usually first simulated by a program that runs one instruction at a time. Even multiprocessors are simulated strictly sequentially like this. Is it possible for a race condition to occur when there are no simultaneous events? Explain.
29. The producer-consumer problem can be extended to a system with multiple producers and consumers that write (or read) to (from) one shared buffer. Assume that each producer and consumer runs in its own thread. Will the solution presented in Fig. 2-28, using semaphores, work for this system?
30. Consider the following solution to the mutual-exclusion problem involving two processes *P0* and *P1*. Assume that the variable *turn* is initialized to 0. Process *P0*'s code is presented below.

```
/* Other code */

while (turn != 0) { /* Do nothing and wait. */
    Critical Section /* . . . */
    turn = 0;

/* Other code */
```

For process *PI*, replace 0 by 1 in above code. Determine if the solution meets *all* the required conditions for a correct mutual-exclusion solution.

31. Show how counting semaphores (i.e., semaphores that can hold an arbitrary value) can be implemented using only binary semaphores and ordinary machine instructions.
32. If a system has only two processes, does it make sense to use a barrier to synchronize them? Why or why not?
33. Can two threads in the same process synchronize using a kernel semaphore if the threads are implemented by the kernel? What if they are implemented in user space? Assume that no threads in any other processes have access to the semaphore. Discuss your answers.
34. Suppose that we have a message-passing system using mailboxes. When sending to a full mailbox or trying to receive from an empty one, a process does not block. Instead, it gets an error code back. The process responds to the error code by just trying again, over and over, until it succeeds. Does this scheme lead to race conditions?
35. The CDC 6600 computers could handle up to 10 I/O processes simultaneously using an interesting form of round-robin scheduling called processor sharing. A process switch occurred after each instruction, so instruction 1 came from process 1, instruction 2 came from process 2, etc. The process switching was done by special hardware, and the overhead was zero. If a process needed T sec to complete in the absence of competition, how much time would it need if processor sharing was used with n processes?
36. Consider the following piece of C code:

```
void main() {  
    fork( );  
    fork( );  
    exit( );  
}
```

How many child processes are created upon execution of this program?

37. Round-robin schedulers normally maintain a list of all runnable processes, with each process occurring exactly once in the list. What would happen (scheduling-wise) if a process occurred twice in the list? Can you think of any reason for allowing this?
38. Can a measure of whether a process is likely to be CPU bound or I/O bound be determined by analyzing source code? How can this be determined at run time?
39. In the section “When to Schedule,” it was mentioned that sometimes scheduling could be improved if an important process could play a role in selecting the next process to run when it blocks. Give a situation where this could be used and explain how.
40. Explain how time quantum value and context switching time affect each other, in a round-robin scheduling algorithm.
41. Measurements of a certain system have shown that the average process runs for a time T before blocking on I/O. A process switch requires a time S , which is effectively

wasted (overhead). For round-robin scheduling with quantum Q , give a formula for the CPU efficiency for each of the following:

- (a) $Q = \infty$
- (b) $Q > T$
- (c) $S < Q < T$
- (d) $Q = S$
- (e) Q nearly 0

42. Five jobs are waiting to be run. Their expected run times are 9, 6, 3, 5, and X . In what order should they be run to minimize average response time? (Your answer will depend on X .)
43. Five batch jobs, A through E , arrive at almost the same time. They have estimated running times of 10, 6, 2, 4, and 8 minutes. Their (externally determined) priorities are 3, 5, 2, 1, and 4, respectively, with 5 being the highest priority. For each of the following scheduling algorithms, determine the mean process turnaround time. Ignore process switching overhead.
- (a) Round robin.
 - (b) Priority scheduling.
 - (c) First-come, first-served (run in order 10, 6, 2, 4, 8).
 - (d) Shortest job first.

For (a), assume that the system is multiprogrammed, and that each job gets its fair share of the CPU. For (b) through (d), assume that only one job at a time runs, until it finishes. All jobs are completely CPU bound.

44. A process running on CTSS needs 30 quanta to complete. How many times must it be swapped in, including the very first time (before it has run at all)?
45. Can you think of a way to save the CTSS priority system from being fooled by random carriage returns?
46. Consider a real-time system with two voice calls of periodicity 5 msec each with CPU time per call of 1 msec, and one video stream of periodicity 33 msec with CPU time per call of 11 msec. Is this system schedulable? Show how you derived your answer.
47. For the above problem, can another video stream be added and have the system still be schedulable?
48. The aging algorithm with $a = 1/2$ is being used to predict run times. The previous four runs, from oldest to most recent, are 40, 20, 40, and 15 msec. What is the prediction of the next time?
49. A soft real-time system has four periodic events with periods of 50, 100, 200, and 250 msec each. Suppose that the four events require 35, 20, 10, and x msec of CPU time, respectively. What is the largest value of x for which the system is schedulable?
50. Explain why two-level scheduling is commonly used. What advantages does it have over single-level scheduling?
51. A real-time system needs to handle two voice calls that each run every 5 msec and consume 1 msec of CPU time per burst, plus one video at 25 frames/sec, with each frame

requiring 20 msec of CPU time. Is this system schedulable? Please explain why or why not it is schedulable and how you came to that conclusion.

52. Consider a system in which it is desired to separate policy and mechanism for the scheduling of kernel threads. Propose a means of achieving this goal.
53. The readers and writers problem can be formulated in several ways with regard to which category of processes can be started when. Carefully describe three different variations of the problem, each one favoring (or not favoring) some category of processes (e.g., readers or writers). For each variation, specify what happens when a reader or a writer becomes ready to access the database, and what happens when a process is finished.
54. Write a shell script that produces a file of sequential numbers by reading the last number in the file, adding 1 to it, and then appending it to the file. Run one instance of the script in the background and one in the foreground, each accessing the same file. How long does it take before a race condition manifests itself? What is the critical region? Modify the script to prevent the race. (*Hint*: use
 In file file.lock
to lock the data file.)
55. Assume that you have an operating system that provides semaphores. Implement a message system. Write the procedures for sending and receiving messages.
56. Rewrite the program of Fig. 2-23 to handle more than two processes.
57. Write a producer-consumer problem that uses threads and shares a common buffer. However, do not use semaphores or any other synchronization primitives to guard the shared data structures. Just let each thread access them when it wants to. Use `sleep` and `wakeup` to handle the full and empty conditions. See how long it takes for a fatal race condition to occur. For example, you might have the producer print a number once in a while. Do not print more than one number every minute because the I/O could affect the race conditions.
58. A process can be put into a round-robin queue more than once to give it a higher priority. Running multiple instances of a program each working on a different part of a data pool can have the same effect. First write a program that tests a list of numbers for primality. Then devise a method to allow multiple instances of the program to run at once in such a way that no two instances of the program will work on the same number. Can you in fact get through the list faster by running multiple copies of the program? Note that your results will depend upon what else your computer is doing; on a personal computer running only instances of this program you would not expect an improvement, but on a system with other processes, you should be able to grab a bigger share of the CPU this way.
59. Implement a program to count the frequency of words in a text file. The text file is partitioned into N segments. Each segment is processed by a separate thread that outputs the intermediate frequency count for its segment. The main process waits until all the threads complete; then it computes the consolidated word-frequency data based on the individual threads' output.